

Politechnika Wrocławska

Wydział Informatyki i Telekomunikacji

Kierunek: **Telekomunikacja (TEL)**

Specjalność: **Sieci teleinformatyczne (TSI)**

PROJEKT ZESPOŁOWY

**Aplikacja do komunikacji głosowej i tekstowej przez sieć
LAN**

Paweł Lipczyk 281330

Michał Gierszon 273182

Konrad Gałka 281277

Kacper Parys 281263

Opiekun projektu:

dr inż. Grzegorz Jaworski

Wrocław 2026

Streszczenie

Celem projektu było stworzenie aplikacji desktopowej umożliwiającej komunikację między użytkownikami w sieci lokalnej (LAN) za pośrednictwem wiadomości tekstowych oraz transmisji głosu. Opracowana aplikacja pozwala na tworzenie pokoju komunikacyjnego na maszynie hosta, do którego mogą dołączać inni użytkownicy znajdujący się w tej samej sieci LAN. Wewnątrz pokoju każdy uczestnik ma możliwość przesyłania wiadomości tekstowych, obrazów oraz plików dowolnego formatu, a także prowadzenia komunikacji głosowej - z możliwością indywidualnej regulacji głośności poszczególnych uczestników lub ich całkowitego wyciszenia. W dalszej części raportu szczegółowo omówiono zastosowane metody implementacyjne wraz z uzasadnieniem podjętych decyzji projektowych.

Abstract

The aim of this project was to develop a desktop application enabling communication between users within a local area network (LAN) through text messaging and voice transmission. The application allows the creation of a communication room on a host machine, which can be joined by other users present within the same LAN. Inside the room, each participant is able to send text messages, images, and files of arbitrary format, as well as engage in real-time voice communication - with support for individual volume control and muting of specific participants. The subsequent sections of this report provide a detailed description of the implementation methods used, along with a justification of the design decisions made throughout the development process.

Spis treści

1	Wstęp	1
1.1	Cel i struktura raportu	1
1.2	Wykorzystanie AI	3
1.3	Opłacalność projektu	4
1.4	Użyteczność społeczna produktu	4
1.5	Plan finansowy projektu	5
1.6	Przegląd rozwiązań dostępnych na rynku	6
2	Backend	8
2.1	Kodek Opus - podstawy teoretyczne	8
2.2	Biblioteka GameNetworkingSockets - podstawy teoretyczne	9
2.3	Obsługa okna graficznego	10
2.4	Architektura aplikacji	11
2.5	Przechwytywanie danych głosowych	12
2.6	Odtwarzanie danych głosowych	15
2.7	Przesyłanie danych tekstowych i plików binarnych	18
2.8	Warstwa sieciowa	19
2.9	Podsumowanie	21
3	Profilowanie i Debugging	23
3.1	Zakres prac i środowisko diagnostyczne	23
3.1.1	Architektura procesu kontroli jakości (QA)	23
3.1.2	Stanowisko testowe i konfiguracja sprzętowa	24
3.1.3	Narzędzia profilujące i diagnostyczne	24
3.1.4	Metodyka badania algorytmu Voice Activity Detection (VAD)	25
3.2	Empiryczne wyniki profilowania i testów	25
3.2.1	Analiza pakietów audio w środowisku sieciowym	25
3.2.2	Weryfikacja poprawności kryptograficznej warstwy transportowej	27
3.2.3	Weryfikacja działania mechanizmu VAD w stanach ciszy	28
3.2.4	Empiryczna weryfikacja zużycia zasobów systemowych	29
3.3	Analiza napotkanych usterek i diagnostyka problemów	29
3.3.1	Usterka regresu jakości audio przy zwiększonej liczbie użytkowników	30
3.3.2	Optymalizacja interfejsu użytkownika i bieżące naprawy kodu	30
3.3.3	Wnioski i podsumowanie prac diagnostycznych	31

Rozdział 1

Wstęp

Dynamiczny rozwój narzędzi do komunikacji zdalnej, zapoczątkowany w latach 90. XX wieku wraz z upowszechnieniem się Internetu, doprowadził do powstania szerokiej gamy komunikatorów tekstowych i głosowych. Współczesne rozwiązania, takie jak Microsoft Teams, Discord czy Slack, oferują rozbudowane funkcjonalności, jednak ich architektura opiera się na centralnych serwerach zewnętrznych dostawców - oznacza to, że przesyłane dane opuszczają sieć lokalną i mogą być przetwarzane przez podmioty trzecie.

W środowiskach wymagających zachowania poufności komunikacji, takich jak sieci korporacyjne, laboratoria badawcze czy instalacje przemysłowe, powyższe rozwiązania mogą stanowić istotne zagrożenie dla bezpieczeństwa informacji. Aplikacja VT-LAN powstała jako odpowiedź na tę potrzebę - jest to komunikator tekstowo-głosowy działający wyłącznie w obrębie sieci lokalnej (LAN), w którym pakiety danych nie są nigdy przekazywane poza jej granice. Kwestia bezpieczeństwa znalazła odzwierciedlenie również na poziomie doboru bibliotek - jako warstwę sieciową zastosowano GameNetworkingSockets (GNS), bibliotekę oferującą wbudowane szyfrowanie transmisji oparte na protokole AES-256-GCM oraz uwierzytelnianie połączeń. Takie podejście zapewnia ochronę przesyłanych danych nie tylko poprzez ograniczenie ruchu do sieci lokalnej, lecz także przez szyfrowanie komunikacji na poziomie aplikacji.

Projekt kładzie nacisk na minimalizm i wydajność - aplikacja charakteryzuje się niskim narzutem na zasoby sprzętowe (CPU, GPU, RAM), prostą obsługą oraz skupieniem na realizacji kluczowych funkcji komunikacyjnych bez zbędnych dodatków. Kod źródłowy aplikacji udostępniono na licencji open source, co umożliwia społeczności jego weryfikację, audyt bezpieczeństwa oraz dalszy rozwój.

1.1. Cel i struktura raportu

Celem raportu jest przedstawienie procesu projektowania i implementacji aplikacji VT-LAN. W raporcie omówiono poszczególne moduły składające się na całość systemu, ze szczególnym uwzględnieniem zastosowanych narzędzi, bibliotek oraz przyjętych decyzji projektowych wraz z ich uzasadnieniem.

Zastosowane technologie

Podczas pracy nad aplikacją VT-LAN zespół korzystał z następujących technologii:

1. **Język C++** - główny i jedyny język wykorzystywany do implementacji logiki aplikacji. Odpowiada zarówno za programowanie interfejsu użytkownika, jak i za obsługę komunikacji sieciowej, przechwytywanie oraz odtwarzanie dźwięku. Wybór C++ podyktowany był wymaganiami wydajnościowymi projektu - język ten zapewnia niski

narzut pamięciowy i pełną kontrolę nad zasobami systemowymi, co jest kluczowe przy przetwarzaniu strumieni audio w czasie rzeczywistym.

2. **Microsoft Visual Studio 2022** - zintegrowane środowisko deweloperskie (IDE) wraz z kompilatorem MSVC, wykorzystywane jako główne narzędzie do budowania i debugowania projektu. Dostarcza zaawansowane narzędzia diagnostyczne [14], w tym debugger z obsługą wielowątkowości oraz zintegrowany pakiet *Diagnostic Tools* umożliwiający profilowanie zużycia CPU, wykonywanie migawek sterty (ang. *heap snapshots*) i lokalizowanie wycieków pamięci w czasie rzeczywistym – bez konieczności modyfikacji kodu produkcyjnego.
3. **Premake5** - system budowania projektów oparty na skryptach Lua. Umożliwia generowanie plików rozwiązania (.sln) dla Visual Studio z pojedynczego, czytelnego pliku konfiguracyjnego. Zastosowanie Premake5 zamiast bezpośredniej edycji plików projektu upraszcza zarządzanie zależnościami oraz ułatwia konfigurację środowiska członkom zespołu - wszelkie zmiany struktury projektu zawarte są w plikach Lua i automatycznie odzwierciedlane przy kolejnym generowaniu rozwiązania.
4. **GitHub** - platforma do współdzielenia kodu źródłowego oparta na systemie kontroli wersji Git. Służyła jako centralne narzędzie do koordynacji prac zespołu - przeglądania zmian w kodzie, udostępniania aktualizacji oraz śledzenia zadań i błędów za pomocą systemu zgłoszeń. Repozytorium pełni również rolę miejsca dystrybucji projektu - użytkownicy mają możliwość wglądu w kod źródłowy aplikacji, a także pobrania skompilowanej wersji wykonywalnej.
5. **Wireshark** [15] - analizator protokołów sieciowych (ang. *packet sniffer*) wykorzystywany podczas prac diagnostycznych i kontroli jakości (QA). Umożliwiał przechwytywanie i inspekcję ruchu sieciowego generowanego przez aplikację, co pozwoliło zweryfikować poprawność działania protokołów UDP i TCP/GNS, skuteczność szyfrowania pakietów audio oraz zachowanie mechanizmu detekcji aktywności głosowej (VAD) – szczególnie opisano w rozdziale 3.
6. **Biblioteki firm trzecich** - projekt korzysta z następujących bibliotek:
 - **SDL3** [1] - wieloplatformowa biblioteka obsługi okna aplikacji, renderowania interfejsu użytkownika oraz systemu audio. W zakresie dźwięku SDL3 dostarcza niskopoziomowe API do otwierania urządzeń wejściowych i wyjściowych, przechwytywania próbek audio z mikrofonu oraz ich odtwarzania przez głośniki. Do miksowania i nakładania strumieni audio aplikacja korzysta z biblioteki **SDL_mixer** [2] - oficjalnego rozszerzenia SDL3 umożliwiającego jednoczesne odtwarzanie wielu kanałów audio z regulacją głośności.
 - **Dear ImGui** [3] - biblioteka interfejsu użytkownika dla języka C++. Odpowiada za renderowanie wszystkich elementów graficznych aplikacji - okien, przycisków, pól tekstowych, list oraz paneli czatu i głosowego. Charakteryzuje się minimalną

liczbą zależności i niskim narzutem wydajnościowym, co czyni ją szczególnie odpowiednią dla aplikacji czasu rzeczywistego. Integruje się bezpośrednio z SDL3 jako backendem okienkowym i wejściowym.

- **Opus** [4] - otwarty kodek audio o niskim opóźnieniu, zoptymalizowany pod transmisję mowy, ze zmiennym bitrate dostosowującym się do warunków sieci. Bezpośrednio integruje się z buforami próbek pobieranymi przez SDL3, kompresując dane przed wysyłką przez GNS i dekompresując je po stronie odbiorcy. Zastosowanie Opus pozwala znacząco zredukować przepustowość wymaganą do transmisji głosu przy zachowaniu wysokiej jakości dźwięku.
- **GameNetworkingSockets (GNS)** [5] - biblioteka sieciowa firmy Valve zapewniająca niezawodną, szyfrowaną komunikację opartą na protokole UDP. Oferuje mechanizmy detekcji opóźnień, kontrolę przepływu oraz opcjonalne gwarancje dostarczenia pakietów, co czyni ją dobrze dopasowaną zarówno do komunikacji klient-serwer, jak i peer-to-peer w sieci LAN. Szyfrowanie realizowane jest przy użyciu algorytmu AES-GCM-256 per pakiet, z wymianą kluczy opartą na Curve25519 [5]. GNS korzysta wewnętrznie z następujących bibliotek:
 - **OpenSSL** [6] - biblioteka kryptograficzna zapewniająca szyfrowanie transmisji oraz uwierzytelnianie połączeń,
 - **protobuf** [7] - biblioteka Google Protocol Buffers wykorzystywana do serializacji wewnętrznych komunikatów sterujących protokołu GNS.

1.2. Wykorzystanie AI

W trakcie realizacji niniejszej pracy projektowej zespół korzystał z narzędzi generatywnej sztucznej inteligencji zgodnie z wytycznymi Wydziału Informatyki i Telekomunikacji Politechniki Wrocławskiej [8]. Wykorzystywane były następujące narzędzia:

- **Claude (Anthropic, wersja Pro),**
- **ChatGPT (OpenAI, wersja Pro),**
- **Gemini (Google).**

Narzędzia te były stosowane wyłącznie jako pomoc techniczna i redakcyjna, w następujących obszarach:

- **Wspomaganie implementacji** – generowanie fragmentów kodu, odpowiedzi dotyczących struktury rozwiązań oraz debugowanie błędów. Fragmenty kodu powstałe przy udziale narzędzi AI zostały oznaczone stosownymi komentarzami bezpośrednio w raporcie,
- **Wyszukiwanie informacji o bibliotekach i technologiach** – zbieranie informacji na temat dostępnych bibliotek, przyjętych standardów oraz rozwiązań stosowanych w branży (m.in. GameNetworkingSockets, Opus, SDL3),

- **Korekta stylistyczna i językowa tekstu** – poprawa poprawności gramatycznej, spójności stylu oraz dostosowanie języka do wymogów raportu.
- **Wsparcie przy diagnostyce i profilowaniu** – w rozdziale dotyczącym profilowania i debuggingu wykorzystano model Gemini (Google) jako asystenta przy ustrukturyzowaniu dokumentu, konsultacjach metodyki testowej dotyczącej filtrów diagnostycznych Wireshark dla strumieni audio opartych na kodeku Opus, oraz przy formułowaniu składni bloków kodu C++ związanych z mechanizmami synchronizacji wątków. Wszelkie wyniki liczbowe, wykresy wydajnościowe oraz analiza błędów opierają się na rzeczywistych testach przeprowadzonych w środowisku laboratoryjnym.

Autorzy pracy zachowali pełną odpowiedzialność za merytoryczną poprawność wszystkich zawartych w niej treści. Każda informacja pozyskana przy udziale narzędzi AI była samodzielnie weryfikowana, w szczególności w zakresie poprawności bibliograficznej i technicznej. Narzędzia AI nie były wykorzystywane do generowania całych rozdziałów ani struktury pracy.

1.3. Oplącalność projektu

VT-LAN jest projektem open source udostępnianym nieodpłatnie, którego model finansowy opiera się na zerowym koszcie utrzymania po stronie autorów. Całość infrastruktury działa wyłącznie na maszynach użytkowników w sieci lokalnej - nie istnieje żaden centralny serwer, który wymagałby hostowania, opłacania ani administrowania. Koszt utrzymania projektu po stronie twórców jest zatem zerowy.

Trwałość projektu zapewniana jest przez społeczność - model open source umożliwia każdemu użytkownikowi zgłaszanie poprawek, łatanie usterek oraz rozwijanie nowych funkcjonalności w formie tzw. patchy. Dzięki temu jakość aplikacji może rosnąć bez ponoszenia przez autorów kosztów deweloperskich. Dodatkowym, dobrowolnym źródłem finansowania mogą być dotacje przekazywane przez użytkowników (np. poprzez platformy typu GitHub Sponsors lub Open Collective), co jest powszechnie stosowaną praktyką w projektach open source.

Biorąc pod uwagę powyższe, projekt cechuje się pełną oplącalnością przy zerowym progu wejścia - zarówno dla autorów, jak i dla użytkowników końcowych.

1.4. Użyteczność społeczna produktu

Rosnąca świadomość zagrożeń związanych z prywatnością danych w komunikatorach internetowych czyni VT-LAN odpowiedzią na realną potrzebę społeczną. Popularne platformy komunikacyjne, takie jak Discord, Teams czy Slack, przesyłają dane głosowe i tekstowe przez zewnętrzne serwery swoich dostawców, co oznacza potencjalną ekspozycję tych danych na osoby trzecie.

Trafnym przykładem ilustrującym tę obawę jest kontrowersja wokół Discorda z początku 2026 roku. Platforma ogłosiła wprowadzenie globalnego systemu weryfikacji wieku, wymagającego od użytkowników przesłania skanu dowodu tożsamości lub wykonania skanu twarzy.

Decyzja ta wywołała falę krytyki ze strony społeczności i organizacji zajmujących się ochroną prywatności - tym bardziej, że we wrześniu 2025 roku doszło do wycieku zdjęć dokumentów tożsamości nawet 70 000 użytkowników Discorda wskutek naruszenia bezpieczeństwa u zewnętrznego dostawcy usług [9]. Skala reakcji była na tyle duża, że Discord oficjalnie opóźnił wdrożenie systemu do drugiej połowy 2026 roku [10].

Incydent ten uwidocznił fundamentalny problem centralnych komunikatorów: dane użytkowników znajdują się poza ich kontrolą. VT-LAN eliminuje ten problem u podstaw - ruch sieciowy nigdy nie opuszcza sieci lokalnej, a szyfrowanie na poziomie aplikacji (AES-GCM-256) dodatkowo chroni przesyłane treści. Aplikacja jest szczególnie użyteczna w środowiskach wymagających poufności, takich jak sieci korporacyjne, laboratoria, instytucje edukacyjne czy domowe sieci prywatne.

Jako projekt open source VT-LAN wpisuje się ponadto w ideę transparentności oprogramowania - każdy użytkownik może zweryfikować kod źródłowy i upewnić się, że aplikacja działa zgodnie z deklarowanymi zasadami. Repozytorium dostępne publicznie w serwisie GitHub umożliwia pobranie dowolnej wersji projektu wraz z pełnym kodem źródłowym i samodzielne skompilowanie go na własnej maszynie. Dzięki temu użytkownicy nie są uzależnieni od decyzji autorów - w przeciwieństwie do zamkniętych platform, takich jak Discord, żadna przyszła aktualizacja nie może bez wiedzy społeczności wprowadzić mechanizmów weryfikacji tożsamości ani przetwarzania danych biometrycznych.

1.5. Plan finansowy projektu

Ze względu na otwartoźródłowy charakter aplikacji oraz jej architekturę opartą wyłącznie na infrastrukturze użytkownika, VT-LAN nie generuje żadnych stałych kosztów operacyjnych po stronie autorów. Poniżej przedstawiono zestawienie kosztów i potencjalnych przychodów projektu.

Koszty

- **Hosting i infrastruktura serwerowa** - brak. Aplikacja działa wyłącznie na maszynach użytkowników w sieci LAN; autorzy nie ponoszą żadnych kosztów serwerowych.
- **Licencje** - brak. Wszystkie zastosowane biblioteki (SDL3, GameNetworkingSockets, Opus, protobuf) udostępniane są na licencjach open source, niewymagających żadnych opłat.
- **Dystrybucja** - brak. Projekt dystrybuowany jest bezpłatnie za pośrednictwem repozytorium GitHub.
- **Utrzymanie** - pokrywane społecznie w modelu open source poprzez zgłoszenia błędów, patche oraz pull requesty. Autorzy mogą hobbystycznie, w miarę dostępnego czasu, weryfikować i zatwierdzać wkład społeczności lub samodzielnie rozwijać projekt.

Przychody

- **Dobrowolne dotacje użytkowników** – projekt może korzystać z platform umożliwiających dobrowolne wspieranie finansowe twórców open source, takich jak GitHub Sponsors lub Open Collective. Przychody z tego źródła mają charakter nieregularny i zależą od zaangażowania społeczności.

Projekt nie jest nastawiony na generowanie zysku. Jego wartość mierzona jest społecznym zasięgiem, liczbą aktywnych użytkowników oraz wkładem społeczności w jego rozwój.

1.6. Przegląd rozwiązań dostępnych na rynku

Na rynku komunikatorów głosowych i tekstowych funkcjonuje kilka ugruntowanych rozwiązań, które można uznać za punkty odniesienia dla projektu VT-LAN. Poniżej przedstawiono ich charakterystykę wraz z oceną pod kątem prywatności, architektury sieciowej i otwartości kodu.

Discord

Discord [9] to jedna z najpopularniejszych platform komunikacyjnych, oferująca czat tekstowy, głosowy oraz wideo. Aplikacja jest bezpłatna w wersji podstawowej, z opcjonalną subskrypcją Discord Nitro.

Z punktu widzenia prywatności Discord jest rozwiązaniem centralnym - cały ruch sieciowy przechodzi przez serwery firmy, co oznacza brak kontroli użytkownika nad przesyłanymi danymi. Kontrowersje wokół próby wprowadzenia przez platformę weryfikacji wieku poprzez skanowanie dokumentu tożsamości i danych biometrycznych, opisane szczegółowo w sekcji 1.4, uwiaryściły systemowe ryzyko wynikające z centralizacji danych [10].

TeamSpeak

TeamSpeak [11] to wieloletnia platforma VoIP skupiona przede wszystkim na komunikacji głosowej, szeroko stosowana w środowiskach graczy oraz w zastosowaniach profesjonalnych. Wyróżnia się niskim opóźnieniem transmisji oraz możliwością uruchomienia własnego serwera, co daje administratorowi pełną kontrolę nad infrastrukturą. Wersja podstawowa jest bezpłatna, natomiast utrzymanie własnego serwera wiąże się z kosztami hostingu. Kod źródłowy pozostaje zamknięty.

Mumble

Mumble [12] jest otwartoźródłowym komunikatorem głosowym o niskim opóźnieniu, działającym w modelu klient-serwer. Serwer (Murmur) może być uruchomiony w sieci lokalnej, co eliminuje konieczność przesyłania danych przez serwery zewnętrzne. Aplikacja jest w pełni bezpłatna i dostępna na licencji open source. Słabą stroną Mumble jest brak natywnej obsługi przesyłania plików.

Microsoft Teams

Microsoft Teams to korporacyjna platforma komunikacyjna zintegrowana z ekosystemem Microsoft 365. Oferuje czat tekstowy, połączenia głosowe i wideo, a także współdzielenie plików i zaawansowane narzędzia pracy zespołowej. Jest rozwiązaniem wyłącznie chmurowym - dane przetwarzane są na serwerach Microsoft, co w środowiskach o podwyższonych wymaganiach bezpieczeństwa może stanowić istotne ograniczenie. Produkt jest płatny w ramach subskrypcji Microsoft 365.

Podsumowanie i pozycja VT-LAN

VT-LAN zajmuje unikalną niszę wśród powyższych rozwiązań. Jest jedynym komunikatorem zaprojektowanym z myślą o działaniu wyłącznie w sieci LAN, łączącym czat tekstowy, transmisję głosową i przesyłanie plików w jednej lekkiej aplikacji open source, niewymagającej żadnej infrastruktury zewnętrznej ani kont użytkowników.

Tabela 1.1. Porównanie komunikatorów dostępnych na rynku

Cecha	Discord	TeamSpeak	Mumble	MS Teams	VT-LAN
Czat tekstowy	Tak	Ograniczony	Ograniczony	Tak	Tak
Głos (VoIP)	Tak	Tak	Tak	Tak	Tak
Przesyłanie plików	Tak	Nie	Nie	Tak	Tak
Tylko LAN	Nie	Nie	Możliwe	Nie	Tak
Open source	Nie	Nie	Tak	Nie	Tak
Własny serwer	Nie	Tak	Tak	Nie	Tak
Bezpłatny	Częściowo	Częściowo	Tak	Nie	Tak
Narzut zasobów	Wysoki	Niski	Niski	Wysoki	Niski

Rozdział 2

Backend

Backend aplikacji VT-LAN stanowi warstwę odpowiedzialną za całą logikę działania systemu - od zarządzania oknem graficznym, przez przechwytywanie i odtwarzanie dźwięku, aż po komunikację sieciową. Jego zadaniem jest integracja trzech niezależnych podsystemów: interfejsu użytkownika opartego na bibliotece ImGui renderowanej przez SDL3, warstwy audio zapewniającej przechwytywanie i odtwarzanie głosu przy użyciu kodeka Opus, oraz warstwy sieciowej zbudowanej na bibliotece GameNetworkingSockets (GNS). Backend pełni rolę łączącą te moduły w spójną całość - przekazuje dane audio do podsystemu sieciowego, odbiera pakiety głosowe i kieruje je do odtwarzacza, a komunikaty tekstowe i pliki binarne przesyła za pomocą niezawodnego kanału GNS.

Poniższy rozdział opisuje kolejno: podstawy teoretyczne kodeka Opus oraz biblioteki GNS, obsługę okna graficznego i przechwytywanie zdarzeń SDL3, architekturę klient-serwer zastosowaną w projekcie, mechanizm przechwytywania i wysyłania danych głosowych wraz z detekcją aktywności mowy (VAD), odtwarzanie odebranych pakietów audio z obsługą utraty pakietów, przesyłanie danych tekstowych i plików binarnych z użyciem mechanizmu RPC, a na końcu szczegóły implementacji warstwy sieciowej GNS.

2.1. Kodek Opus - podstawy teoretyczne

Geneza i standaryzacja

Opus [4] to otwarty kodek audio opracowany przez grupę roboczą IETF i opublikowany w 2012 roku jako RFC 6716. Powstał z połączenia dwóch niezależnych projektów: kodeka SILK rozwijanego przez Skype od 2007 roku, zoptymalizowanego pod transmisję mowy, oraz kodeka CELT opracowanego przez fundację Xiph.Org, przeznaczonego do transmisji dźwięku ogólnego przeznaczenia przy bardzo niskim opóźnieniu [13]. Połączenie obu komponentów pozwoliło uzyskać kodek pokrywający szeroki zakres zastosowań - od wąskopasmowej transmisji mowy przy 6 kbps aż po wysokiej jakości stereofoniczny dźwięk przy 510 kbps [4].

Architektura hybrydowa - SILK i CELT

Koder Opus składa się z dwóch głównych warstw operujących na różnych reprezentacjach sygnału [4]:

- **Warstwa LP (SILK)** - oparta na liniowym przewidywaniu (ang. *Linear Prediction*). Modeluje sygnał mowy jako pobudzenie filtra autoregresyjnego, co jest wydajne obliczeniowo i daje dobre wyniki dla mowy przy niskim bitrate. Stosowana w trybie SILK-only dla połączeń do ok. 32 kbps.

- **Warstwa MDCT (CELT)** - oparta na zmodyfikowanej dyskretnej transformacji kosinusowej (ang. *Modified Discrete Cosine Transform*). Koduje sygnał w dziedzinie częstotliwości, co pozwala na bardzo niskie opóźnienie algorytmiczne (od 2,5 ms na ramkę) i wysoką jakość dla sygnałów niewokalnych. Stosowana samodzielnie w trybie CELT-only lub łączona z SILK w trybie hybrydowym.

RFC 6716 definiuje 32 możliwe konfiguracje kodeka, różniące się trybem pracy, pasmem audio i rozmiarem ramki [4]. W VT-LAN stosowany jest tryb SILK-only lub hybrydowy z ramką 20 ms i bitrate 32 kbps, co odpowiada konfiguracji zoptymalizowanej dla VoIP.

Packet Loss Concealment (PLC)

Istotną właściwością dekodera Opus z punktu widzenia transmisji sieciowej jest wbudowany mechanizm PLC (ang. *Packet Loss Concealment*). W przypadku wywołania `opus_decode` z pustym wejściem (`nullptr`, `0`) dekodery nie zwraca ciszy, lecz ekstrapoluje brakujący fragment sygnału na podstawie ostatnio zdekodowanych ramek. Dla sygnału mowy, który ma silną korelację czasową, algorytm PLC potrafi wiarygodnie odtworzyć krótkie ubytki rzędu 20-60 ms bez subiektywnie odczuwalnych artefaktów [4].

Variable Bit Rate (VBR)

Opus wspiera zarówno stały (CBR), jak i zmienny bitrate (VBR). W trybie VBR koder dynamicznie przydziela więcej bitów ramkom zawierającym złożone fragmenty sygnału (spółgłoski, fragmenty szumowe), a mniej - ramkom zawierającym proste segmenty (samogłoski, cisza). Przekłada się to na zmniejszenie średniego zużycia pasma przy zachowaniu subiektywnej jakości dźwięku [4]. W VT-LAN VBR jest włączony z docelowym bitrate 32 kbps.

2.2. Biblioteka GameNetworkingSockets - podstawy teoretyczne

Geneza i przeznaczenie

GameNetworkingSockets (GNS) [5] to biblioteka warstwy transportowej pierwotnie opracowana przez firmę Valve Corporation na potrzeby platformy Steam, a następnie udostępniona jako projekt open source. Jej głównym zadaniem jest dostarczenie programiście interfejsu opartego na modelu komunikatów (ang. *message-oriented*), który ukrywa złożoność protokołu UDP, jednocześnie zachowując jego niskie opóźnienie.

Model niezawodności

GNS rozszerza UDP o warstwę niezawodności wzorowaną na mechanizmach protokołu QUIC oraz algorytmie *ack vector* z RFC 4340 [5]. Odbiorca informuje nadajnika o statusie każdego odebranego pakietu, dzięki czemu retransmitowane są wyłącznie brakujące segmenty - bez

czekania na wygaśnięcie timeoutu jak w TCP. Wiadomości mogą przekraczać rozmiar MTU, a fragmentacja i składanie pakietów obsługiwane są transparentnie przez bibliotekę [5].

Tryby wysyłania wiadomości

GNS udostępnia kilka trybów dostarczenia wiadomości, wybieranych przez zestaw flag przekazywanych do funkcji wysyłającej [5]:

- **Reliable** - gwarantowane dostarczenie z zachowaniem kolejności, analogiczne do TCP. Stosowane dla wiadomości tekstowych, transferów plików i sygnałów sterujących.
- **Unreliable** - wysyłka bez gwarancji dostarczenia ani zachowania kolejności. Stosowane dla pakietów głosowych, gdzie opóźnienie jest ważniejsze niż niezawodność.
- **UnreliableNoNagle** - jak wyżej, ale z wyłączonym algorytmem Nagle'a, który w standardowym TCP buforuje małe pakiety przed wysłaniem w celu zwiększenia efektywności. Wyłączenie buforowania redukuje opóźnienie kosztem nieznacznie wyższego narzutu sieciowego.

Szacowanie przepustowości

GNS implementuje mechanizm kontroli przepływu oparty na RFC 5348 (TCP-friendly Rate Control, TFRC) [5]. Biblioteka na bieżąco szacuje dostępną przepustowość połączenia i udostępnia te informacje aplikacji przez strukturę statystyk połączenia, umożliwiając adaptacyjne zarządzanie strumieniem danych - w VT-LAN wykorzystywane przy rate-limitingu segmentów pliku.

2.3. Obsługa okna graficznego

Biblioteka SDL3

Simple DirectMedia Layer (SDL) [1] to wieloplatformowa biblioteka napisana w języku C, zapewniająca interfejs do sprzętu systemu operacyjnego: okien, renderingu, wejścia (klawiatura, mysz, kontrolery) oraz urządzeń audio.

Powody wyboru SDL3 jako podstawy okna i systemu audio w VT-LAN:

- **Niski narzut zasobów** - SDL3 jest biblioteką niskopoziomową, w której programista zachowuje pełną kontrolę nad alokacją pamięci i cyklem życia obiektów. W połączeniu z C++ przekłada się to na minimalne zużycie RAM oraz niskie obciążenie procesora. Jest to szczególnie istotne przy równoczesnym przetwarzaniu strumienia audio w czasie rzeczywistym, obsłudze połączeń sieciowych oraz renderowaniu interfejsu użytkownika - wszystkich wykonywanych jednocześnie w osobnych wątkach.
- **Dostęp do urządzeń audio systemu operacyjnego** - SDL3 udostępnia jednolite API do otwierania fizycznych urządzeń nagrywających i odtwarzających, niezależnie od platformy systemowej. Programista operuje na uchwycie strumienia audio

(SDL_AudioStream), który zachowuje się jak kolejka FIFO próbek z wbudowaną konwersją formatu i częstotliwości próbkowania - bez konieczności ręcznej obsługi tych przekształceń.

- **Wieloplatfromowość** - ten sam kod źródłowy kompiluje się na systemach Windows, Linux i macOS bez modyfikacji warstwy systemowej. Ewentualna kompilacja VT-LAN na innej platformie sprowadza się wyłącznie do zmian w logice aplikacji, nie w kodzie obsługi urządzeń czy okna.

Przechwytywanie zdarzeń - drag and drop

SDL3 dostarcza pętlę zdarzeń (SDL_PollEvent), z której aplikacja pobiera wszystkie zdarzenia systemowe: klawiaturę, mysz, zmianę rozmiaru okna, a także zdarzenia przeciągnij-i-upuść (*drag and drop*).

W VT-LAN obsługa upuszczania pliku na okno aplikacji zaimplementowana jest poprzez rejestrację callbacku na zdarzeniu SDL_EVENT_DROP_FILE:

```
1 SetEventCallback([](SDL_Event event) {
2     ImGui_ImplSDL3_ProcessEvent(&event);
3     if (event.type == SDL_EVENT_DROP_FILE) {
4         static_cast<Lobby*>(StateMachine.GetTop())
5             ->OnDropFile(event.drop.data);
6     }
7 });
```

Listing 2.1. Obsługa zdarzenia drag-and-drop w SDL3 (praca własna)

Pole `event.drop.data` zawiera ścieżkę do upuszczonego pliku. Wywołanie `OnDropFile` inicjuje procedurę przesyłania pliku opisaną w sekcji 2.7.

2.4. Architektura aplikacji

Model klient-serwer z hostem zintegrowanym

VT-LAN przyjmuje klasyczny model klient-serwer, z jedną istotną różnicą: instancja serwera nie jest odrębnym procesem ani dedykowaną maszyną - jeden z uczestników sesji jednocześnie pełni rolę hosta (serwera) i klienta. Jest to popularny wzorzec w grach wieloosobowych (*listen server*), który eliminuje konieczność utrzymywania dedykowanej infrastruktury serwerowej w sieci LAN. Model ten dobrze wpisuje się w założenia projektu - użytkownik uruchamia pojedynczą aplikację, tworzy pokój lub dołącza do istniejącego, i może od razu rozpocząć komunikację bez konieczności konfiguracji dodatkowego oprogramowania.

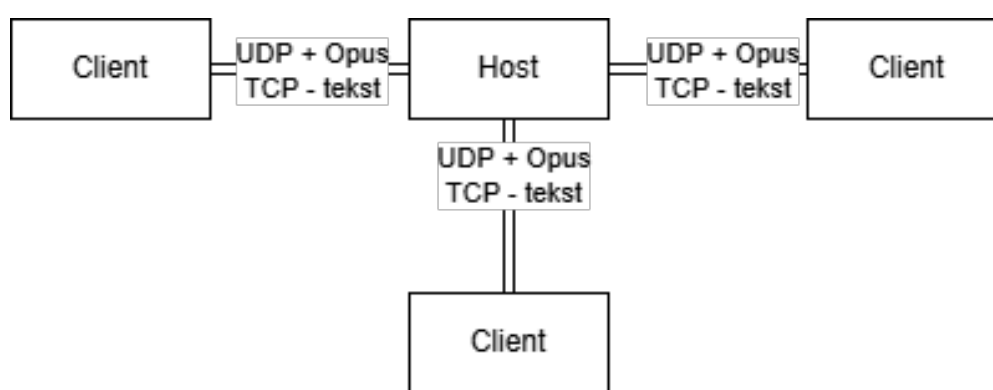
Podejście to ma następujące konsekwencje:

- Host otwiera gniazdo nasłuchujące (`CreateListenSocketIP`) na wybranym porcie UDP i jest osiągalny przez pozostałych uczestników sieci lokalnej po adresie IP.
- Klienci łączą się do hosta wywołując `ConnectByIPAddress`.

- Cały ruch przechodzi przez hosta: wiadomości tekstowe i pliki wysyłane przez klienta trafiają najpierw do hosta, który je przekazuje do pozostałych uczestników. Pakiety głosowe są przekazywane analogicznie, jednak przez dedykowany mechanizm opisany w sekcji 2.6.
- Zakończenie sesji przez hosta powoduje rozłączenie wszystkich klientów - brak hosta oznacza brak sesji.

Diagram przepływu danych

Poniżej przedstawiono uproszczony schemat przepływu danych między uczestnikami sesji VT-LAN. Wyróżnione zostały dwie ścieżki transmisji: głosowa (UDP, bez gwarancji dostarczenia) oraz tekstowo-plikowa (GNS reliable, zachowanie kolejności).



Rysunek 2.1. Schemat przepływu danych w architekturze VT-LAN (praca własna)

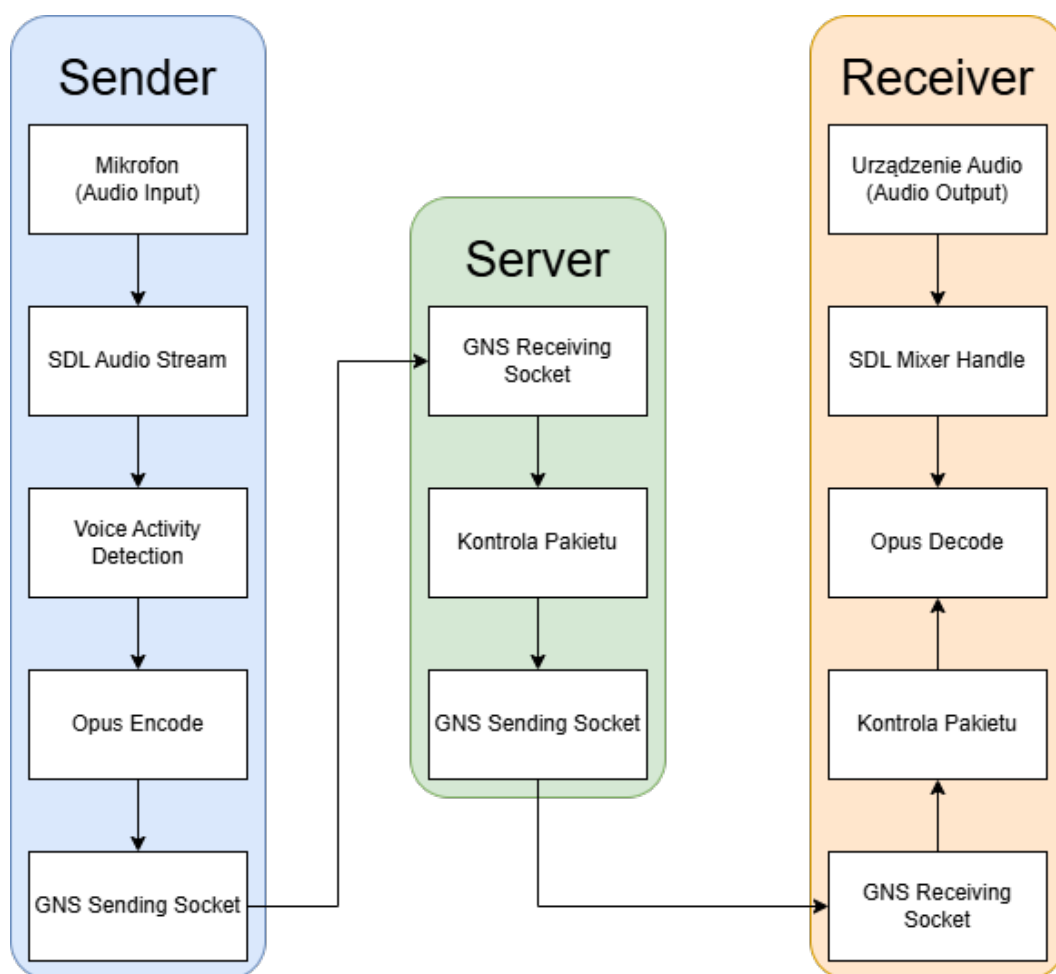
Tabela 2.1. Porównanie kanałów transmisji w VT-LAN

Cecha	Głos (unreliable)	Tekst (reliable)	Pliki (reliable)
Protokół	UDP (GNS unreliable)	UDP z gwarancją GNS	UDP z gwarancją GNS
Gwarancja dostarczenia	Nie	Tak	Tak
Zachowanie kolejności	Nie	Tak	Tak
Priorytet	Wysoki	Normalny	Niski
Typ danych	Zakodowane ramki Opus	Ciągi znaków	Segmenty binarne
Obsługa utraty pakietu	Opus PLC (Packet Loss Concealment)	Retransmisja przez GNS	Retransmisja przez GNS

2.5. Przechwytywanie danych głosowych

Przegląd "pipeline'u" transmisji głosowej

Poniższy schemat przedstawia kompletną ścieżkę przetwarzania dźwięku w aplikacji VT-LAN - od przechwycenia sygnału z mikrofonu po jego odtworzenie u odbiorcy. Kolejne podsekcje opisują szczegółowo poszczególne etapy tego procesu.



Rysunek 2.2. "Pipeline" transmisji głosowej w VT-LAN: nadawca, serwer i odbiorca (praca własna)

Inicjalizacja urządzenia nagrywającego

SDL3 umożliwia otwieranie urządzeń audio przez `SDL_OpenAudioDevice` (nagrywanie) i zwraca `SDL_AudioStream` - strumień, z którego aplikacja pobiera próbki poleceniem `SDL_GetAudioStreamData`. W VT-LAN urządzenie domyślne jest otwierane przy starcie aplikacji z następującą specyfikacją:

```

1 SDL_AudioSpec src{};
2 src.freq      = 48000;           // 48 kHz - standard Opus
3 src.format    = SDL_AUDIO_F32;  // próbki zmiennoprzecinkowe 32-bit
4 src.channels  = 1;              // mono
5
6 audioDeviceManager.OpenDefaultDevice(AudioDeviceKind::Recording, &
   src);
  
```

Listing 2.2. Inicjalizacja urządzenia nagrywającego (praca własna)

Parametry 48 kHz, format F32 i mono wynikają bezpośrednio z wymagań kodera Opus, który operuje w tym standardzie dla trybu VoIP [4].

Wątek przechwytywania - VoiceCaptureManager

Przechwytywanie audio odbywa się w dedykowanym wątku (`m_voiceCaptureThread`), uruchamianym przez `VoiceCaptureManager::Start()`. Wątek co 1 ms odpytuje strumień wejściowy wywołaniem `SDL_GetAudioStreamData`, pobierając dostępne próbki PCM i dołączając je do wewnętrznego bufora `m_sendVoiceBuffer`. Bufor ten chroniony jest przez `std::mutex`, ponieważ dostęp do niego współdzielony jest z wątkiem sieciowym odpowiedzialnym za kodowanie i wysyłkę danych. Zastosowanie osobnego wątku przechwytywania gwarantuje, że opóźnienia po stronie sieci lub kodeka nie wpływają na regularność pobierania próbek z urządzenia wejściowego.

Voice Activity Detection (VAD)

Przed zakodowaniem i wysłaniem ramki audio sprawdzane jest, czy zawiera ona faktyczną aktywność głosową. Mechanizm detekcji aktywności mowy (VAD, ang. *Voice Activity Detection*) oparty jest na pomiarze RMS (ang. *Root Mean Square*) - pierwiastka ze średniej kwadratów amplitud próbek w ramce:

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_{i=1}^N s_i^2} \quad (2.1)$$

gdzie:

- s_i - wartość i -tej próbki w ramce,
- N - liczbę próbek.

Jeśli RMS jest niższy niż wartość progowa (`m_rmsThreshold`), ramka uznawana jest za ciszę. Aby uniknąć urywania głosu przy chwilowych spadkach głośności, zaimplementowany jest mechanizm *grace period*: ciche ramki są odrzucane dopiero po przekroczeniu licznika `IS_SILENT_GRACE_FRAMES` kolejnych cichych klatek.

```
1 bool VoiceSendingManager::IsSilent(const float* frame, size_t
   samples)
2 {
3     if (samples == 0) return true;
4
5     double sumSquares = 0.0;
6     for (size_t i = 0; i < samples; ++i) {
7         float s = frame[i];
8         sumSquares += static_cast<double>(s) * static_cast<double>(
           s);
9     }
10    float rms = static_cast<float>(std::sqrt(sumSquares / samples))
       ;
11
12    if (rms >= m_rmsThreshold) {
```

```

13     m_currentSilentFrames = 0;
14     return false;
15 }
16
17 if (m_currentSilentFrames < IS_SILENT_GRACE_FRAMES) {
18     m_currentSilentFrames++;
19     return false;
20 }
21 return true;
22 }

```

Listing 2.3. Implementacja VAD oparta na RMS (praca własna)

Zakończenie aktywności mowy sygnalizowane jest przez wysłanie ostatniej ramki ciszy z numerem sekwencji ustawionym na wartość 1 (`m_nextSeq = 1`). Odbiorca interpretuje tę wartość jako sygnał resetu sekwencji - mechanizm ten opisano dokładniej w sekcji 2.6. Pozwala to zresetować wewnętrzny stan dekodera Opus po każdej przerwie w transmisji, co eliminuje artefakty dźwiękowe na początku kolejnego fragmentu mowy.

Kodowanie Opus i kolejkovanie ramek

Każda ramka 960 próbek (odpowiadająca 20 ms przy 48 kHz) jest kompresowana koderem Opus skonfigurowanym w trybie VoIP z bitrate 32 kbps i aktywnym VBR (ang. *Variable Bit Rate*):

```

1 m_encoder = opus_encoder_create(VOICE_SAMPLE_RATE, 1,
2                                OPUS_APPLICATION_VOIP, &err);
3 opus_encoder_ctl(m_encoder, OPUS_SET_BITRATE(32000));
4 opus_encoder_ctl(m_encoder, OPUS_SET_VBR(1));

```

Listing 2.4. Inicjalizacja kodera Opus (praca własna)

Zakodowane ramki trafiają do kolejki `m_encodedFrames`, a wątek wysyłający wywołuje `sender.SendVoicePacket` dla każdej z nich, oznaczając je numerem sekwencji (`uint16_t`), identyfikatorem nadawcy (`uint8_t`) i flagą nagłówkową `VoicePacket`. Pakiety głosowe wysyłane są z flagą `k_nSteamNetworkingSend_UnreliableNoNagle`, co zapewnia minimalny narzut i najniższe możliwe opóźnienie transmisji.

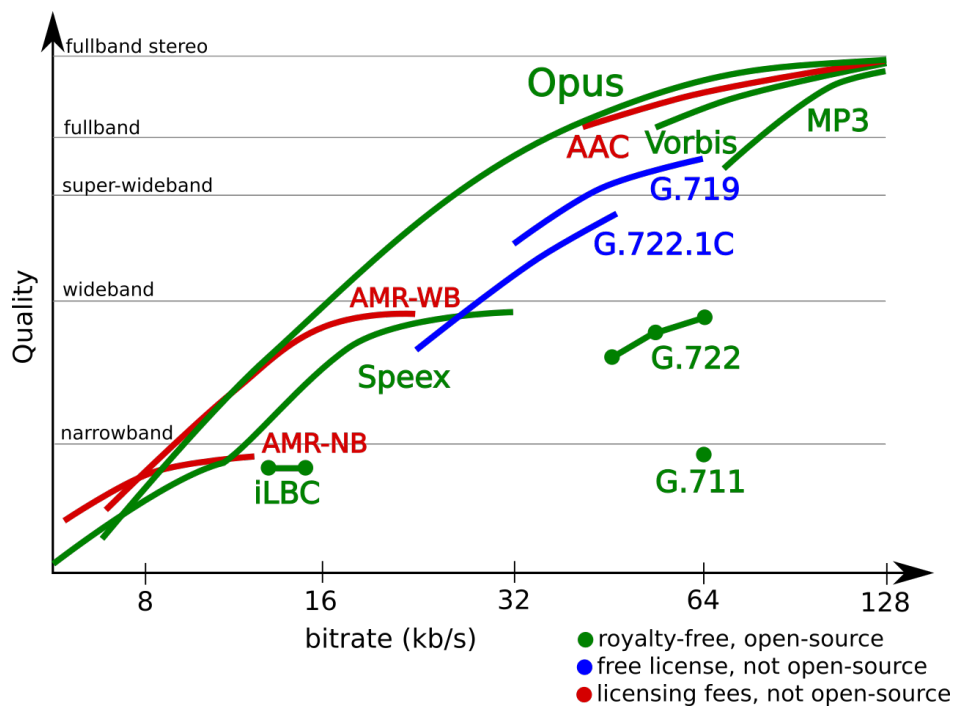
2.6. Odtwarzanie danych głosowych

Kodek Opus - strona odbiorcza

Szczegółowy opis architektury i właściwości kodeka Opus przedstawiono w sekcji 2.1. Właściwości kodeka, które zdecydowały o jego wyborze w projekcie VT-LAN, zestawiono w tabeli 2.2.

Tabela 2.2. Wybrane parametry kodeka Opus [4]

Parametr	Wartość / opis
Wspierane bitrate	6-510 kbps (VBR/CBR)
Wspierane częstotliwości próbkowania	8, 12, 16, 24, 48 kHz
Rozmiar ramki	2,5-60 ms
Opóźnienie algorytmiczne	26,5 ms (tryb VoIP)
Obsługa PLC	Tak (wbudowana w opus_decode)
Licencja	BSD, royalty-free



Rysunek 2.3. Porównanie jakości kodeka Opus z innymi kodekami w funkcji przepustowości (źródło: opus-codec.org)

Wątek odtwarzania - VoiceReceivingManager

Odtwarzanie głosu realizowane jest przez VoiceReceivingManager działający na osobnym wątku. Wątek w każdej iteracji pętli wykonuje dwie operacje: odbiera z bufora sieciowego nowe zakodowane ramki (PollAndProcessMessages) i wywołuje PlayAll dla każdego aktywnego nadawcy.

Dla każdego uczestnika sesji przechowywana jest struktura PlayerVoiceData zawierająca:

- dekodery Opus (OpusDecoder*),
- strumień audio SDL3 (SDL_AudioStream*),
- ścieżkę SDL_mixer (MIX_Track*) dla miksowania z regulacją głośności i opcjonalnym dźwiękiem przestrzennym,

- mapę oczekujących pakietów indeksowaną numerem sekwencji (`std::map<uint16_t, EncodedFrame>`).

Kolejkowanie i obsługa utraty pakietów (PLC)

Odebrane pakiety są wstawiane do mapy `m_pendingVoicePackets` z kluczem odpowiadającym numerowi sekwencji. W każdym kroku wątek odtwarzania sprawdza, czy oczekiwany numer sekwencji (`m_expectedSeq`) jest dostępny w mapie:

1. **Ramka dostępna** - dekodowanie normalne przez `opus_decode`, próbki trafiają do strumienia SDL3.
2. **Ramka niedostępna, następna jest w zasięgu ($\text{gap} \leq 3$)** - *Packet Loss Concealment* (PLC): `opus_decode` wywoływany jest z pustym wejściem (`nullptr, 0`), co uruchamia wbudowany algorytm ekstrapolacji PLC kodera Opus interpolujący brakujące próbki.
3. **Ramka niedostępna, luka jest duża** - stan dekodera i strumień SDL3 są resetowane, a odtwarzanie rozpoczyna się od najbliższego dostępnego pakietu.

```
1 int plcSamples = opus_decode(
2     playerData.m_decoder,
3     nullptr, 0,          // brak danych - PLC
4     plcBuf,
5     kMaxFrameSamples,
6     0
7 );
```

Listing 2.5. PLC - wywołanie dekodera Opus z pustym wejściem (praca własna)

Reset stanu dekodera po przerwie w mowie wyzwalany jest odbiorem ramki z numerem sekwencji 1 (umowny sygnał końca aktywności mowy, wysyłany przez stronę nadawczą po wykryciu ciszy przez VAD):

```
1 if (sequenceID == 1) {
2     opus_decoder_ctl(playerData.m_decoder, OPUS_RESET_STATE);
3     SDL_ClearAudioStream(playerData.m_stream);
4     playerData.m_pendingVoicePackets.clear();
5     playerData.m_expectedSeq = 2;
6 }
```

Listing 2.6. Reset dekodera po wykryciu przerwy w mowie (praca własna)

Przekazywanie pakietów głosowych przez serwer

Ponieważ w modelu *listen server* wszystkie pakiety przechodzą przez hosta, `VoiceReceivingManager` na serwerze po odebraniu pakietu głosowego od klienta retransmituje go do wszystkich pozostałych klientów (`ForwardFrameData`), usuwając z maski odbiorców zarówno hosta, jak i nadawcę oryginalnego pakietu.

2.7. Przesyłanie danych tekstowych i plików binarnych

Mechanizm RPC

Komunikacja tekstowa i transfery plików realizowane są przy użyciu własnego mechanizmu zdalnych wywołań procedur (RPC, ang. *Remote Procedure Call*). Każde RPC rejestrowane jest statycznie przy starcie aplikacji przez obiekt `RPCRegistrar`, który przypisuje mu unikalny identyfikator 16-bitowy, poziom uprawnień (*Authority*) oraz funkcję typu lambda wywoływaną po odebraniu.

```
1 static fs::RPCRegistrar reg_chat(  
2     "ChatBroadcast",  
3     fs::Authority::All,  
4     [](uint8_t senderID, std::string text) {  
5         auto self = fs::connections.GetSelf();  
6         if (self.IsValid() && self.GetCustomID() == senderID)  
7             return; // ignoruj echo własnej wiadomości  
8         chatManager.Enqueue(senderID, text);  
9     }  
10 );
```

Listing 2.7. Rejestracja RPC nadawania wiadomości czatu (praca własna)

Poziom uprawnień `Authority::All` oznacza, że zarówno serwer, jak i klienci mogą inicjować wywołanie. Klient wysyła *request* do serwera, który po weryfikacji uprawnień przekazuje wiadomość do pozostałych uczestników (`ForwardRPCRequest`).

Sygnały sterujące - hasło i inicjalizacja sesji

Przed przystąpieniem do sesji serwer może wymagać podania hasła. Weryfikacja odbywa się przez dedykowane RPC wywoływane tuż po nawiązaniu połączenia TCP-like. Jeśli hasło jest niepoprawne, serwer zamyka połączenie z kodem przyczyny (`m_eEndReason`). Inicjalizacja nowego uczestnika przebiega następująco:

1. Serwer otrzymuje żądanie połączenia (`ConnectionStateConnecting`).
2. Po pomyślnym nawiązaniu (`ConnectionStateConnected`) wysyła do nowego uczestnika RPC `NewPlayerSync` z listą wszystkich aktualnych połączeń.
3. Do pozostałych uczestników wysyłane jest RPC `AddPlayerSync` z danymi nowego klienta (identyfikator, nazwa wyświetlana).

Przesyłanie plików

Transfer pliku podzielony jest na dwie fazy realizowane przez `FileTransferManager`:

1. **Inicjacja** - RPC `FileTransferStart` przekazuje metadane: identyfikator transferu (32-bit), identyfikator nadawcy, nazwę pliku, rozmiar całkowity i liczbę chunków.

2. Wysyłanie segmentów - plik dzielony jest na kawałki o stałym rozmiarze (SEGMENT_SIZE).

Aby nie przepełnić bufora nadawczego GNS (ok. 512 KB na połączenie), segmenty wysyłane są z ograniczeniem przepustowości przez metodę `Tick` wywoływana co 16 ms z limitem `SEGMENTS_PER_TICK` segmentów na iterację.

Po stronie odbiorczej segmenty gromadzone są w mapie indeksowanej numerem segmentu. Gdy wszystkie fragmenty zostaną odebrane, `TryAssembleLocked` łączy je w całość i zapisuje plik w katalogu `received_files/`, dodając sufiks w przypadku konfliktu nazwy.

Tabela 2.3. Struktura pakietu sieciowego wysyłającego segment pliku

Pole	Rozmiar	Opis
Nagłówek sieciowy	1 B	<code>NetworkingHeader::RPCCall</code>
ID RPC	2 B	Identyfikator <code>FileTransferChunk</code>
Transfer ID	4 B	Unikalny identyfikator transferu
Segment index	4 B	Numer porządkowy fragmentu
Dane	do <code>SEGMENT_SIZE</code> B	Surowe bajty fragmentu pliku

2.8. Warstwa sieciowa

GameNetworkingSockets

GameNetworkingSockets (GNS) [5] to biblioteka sieciowa rozwijana przez firmę Valve Corporation, pierwotnie wbudowana w platformę Steam i udostępniona jako projekt open source. GNS opiera się na protokole UDP, rozszerzając go o dodatkowe mechanizmy dostępne w zależności od trybu wysyłania:

- **Niezawodna kolejkowana transmisja** (analogiczna do TCP) - gwarancja dostarczenia i zachowanie kolejności pakietów przy minimalnym narzucie; używana dla wiadomości tekstowych, RPC i segmentów pliku.
- **Transmisja bez gwarancji (unreliable)** - flaga `k_nSteamNetworkingSend_UnreliableNoNagle` wyłącza oczekiwanie na potwierdzenia; stosowana dla pakietów głosowych, gdzie opóźnienie jest ważniejsze niż niezawodność.

Kanały i mechanizm `ConnectionMessageFlag`

Aby uniknąć iteracji po wszystkich aktywnych połączeniach przy każdej operacji wysyłania, VT-LAN wprowadza własną abstrakcję flag bitowych `ConnectionMessageFlag`. Każdy uczestnik sesji otrzymuje unikalny identyfikator (`CustomID`), któremu odpowiada bit w 16-bitowej masce. Operacje na maskach pozwalają szybko określić zestaw odbiorców:

- `AllCurrentReceivers()` - maska wszystkich aktualnie połączonych uczestników,
- `ClearFlag(mask, flag)` - usunięcie konkretnego uczestnika z maski (np. usunięcie nadawcy przed przekazaniem wiadomości),

- `HostFlag()`, `MeFlag()` - predefiniowane flagi hosta i samego siebie.

`AllCurrentReceivers` i `MeFlag` są atomowymi zmiennymi `std::atomic<uint16_t>`, co zapewnia bezpieczny odczyt z wielu wątków bez blokad.

System RPC - przekazywanie i uprawnienia

Pakiet RPC ma strukturę:

- [nagłówek 1B],
- [ID RPC 2B],
- [argumenty].

Argumenty konwertowane są do postaci binarnej i dołączane bezpośrednio do bufora. Na serwerze po odebraniu pakietu z typem `NetworkingHeader::RPCCall` wywoływana jest zarejestrowana funkcja lambda, a następnie - jeśli RPC ma poziom `Authority::All` - wiadomość przekazywana jest do pozostałych odbiorców przez `ForwardRPCRequest`.

Callbacks stanu połączenia

GNS informuje aplikację o zmianach stanu połączenia przez globalny callback `SteamNetConnectionStatusChangedCallback_t`. W VT-LAN zarejestrowany jest handler `NetworkingGNS::HandleConnectionHelper`, który deleguje callbacki i na podstawie wartości `m_info.m_eState` wywołuje jedną z metod:

Tabela 2.4. Obsługiwane stany połączenia GNS

Stan GNS	Akcja w VT-LAN
Connecting	Akceptacja połączenia (serwer) lub rejestracja hosta (klient)
Connected	Wysłanie RPC synchronizacji listy graczy
ClosedByPeer	Usunięcie uczestnika, powiadomienie pozostałych przez RPC
ProblemDetectedLocally	Jak wyżej, dodatkowo wywołanie callbacku utraty połączenia

Podział ruchu - priorytety strumieni

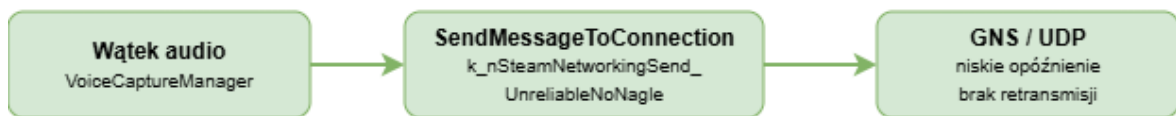
GNS udostępnia API komunikatów sieciowych (*Steam Networking Messages API*), w którym każda wiadomość może być opatrzona innym zestawem flag dostarczenia. VT-LAN wykorzystuje to do rozdzielenia ruchu na dwa niezależne strumienie:

- **Pakiety głosowe** wysyłane są z flagą `k_nSteamNetworkingSend_UnreliableNoNagle` - pakiet trafia do sieci natychmiast po każdej iteracji wątku audio, bez buforowania i bez retransmisji. Priorytetem jest minimalne opóźnienie kosztem gwarancji dostarczenia, co jest akceptowalne w przypadku strumienia głosowego.

- **Pozostałe pakiety** (wiadomości tekstowe, pliki, sygnały sterujące) wysyłane są z domyślnymi flagami niezawodności przez wywołanie `Sender::Flush`, które następuje raz na klatkę logiki aplikacji. Gwarantuje to zachowanie kolejności i pewność dostarczenia kosztem nieznacznie wyższego opóźnienia.

Taka architektura zapewnia, że przesyłanie pliku nie wpływa na jakość transmisji głosowej - oba strumienie konkurują o przepustowość niezależnie, dzięki czemu użytkownik może prowadzić rozmowę i jednocześnie wysyłać duże pliki bez zauważalnego wzrostu opóźnienia audio.

Strumień głosowy (priorytetowy)



Strumień danych (standardowy)



Rysunek 2.4. Podział ruchu sieciowego na priorytetowy strumień głosowy i standardowy strumień danych (praca własna)

2.9. Podsumowanie

Niniejszy rozdział opisał implementację warstwy backendowej aplikacji VT-LAN. Centralną rolę pełni SDL3, dostarczający zarówno okno graficzne z pętlą zdarzeń - umożliwiającą m.in. obsługę przeciągania i upuszczania plików (ang. *drag-and-drop*) - jak i niskopoziomowe API urządzeń audio. Przechwytywanie dźwięku realizowane jest w dedykowanym wątku, co gwarantuje ciągłość strumienia niezależnie od obciążenia wątku głównego.

Moduł detekcji aktywności mowy oparty na pomiarze RMS eliminuje zbędne ramki ciszy, redukując zużycie pasma i zapewniając poprawne resetowanie stanu dekodera Opus po stronie odbiorczej. Odtwarzanie głosu korzysta z wbudowanego mechanizmu PLC kodeka Opus, który minimalizuje wpływ utraty pakietów na jakość odbieranego dźwięku.

Architektura *listen server* eliminuje konieczność utrzymywania dedykowanej infrastruktury sieciowej, a mechanizm flag bitowych `ConnectionMessageFlag` upraszcza zarządzanie zestawem odbiorców przy minimalnym narzucie pamięciowym. Komunikacja tekstowa i transfer plików realizowane są przez własny mechanizm RPC z obsługą uprawnień oraz rate-limitingiem chunków, zapobiegającym przepełnieniu buforów sieciowych.

Fizyczny podział ruchu na bezgwarancyjne pakiety głosowe i niezawodne pakiety danych sprawia, że jednoczesna rozmowa i transfer pliku nie powodują wzajemnych zakłóceń. Całość

komunikacji szyfrowana jest automatycznie przez bibliotekę GNS z użyciem algorytmu AES-GCM-256.

Rozdział 3

Profilowanie i Debugging

Niniejszy rozdział opisuje działania z zakresu diagnostyki, profilowania wydajnościowego oraz zapewnienia jakości aplikacji VT-LAN. Omówiono w nim metodykę projektowania metryk jakościowych, profilowanie zużycia zasobów sprzętowych, monitorowanie przepustowości sieciowej, prowadzenie bazy błędów oraz ich lokalizację i eliminację w kodzie źródłowym.

3.1. Zakres prac i środowisko diagnostyczne

W ramach procesu kontroli jakości aplikacji VT-LAN zrealizowano następujące działania:

- Zaprojektowanie i wdrożenie metryk jakościowych oraz planu testów obciążeniowych dla modułu audio i czatu.
- Profilowanie zużycia zasobów sprzętowych (CPU oraz RAM) w celu eliminacji wycieków pamięci i optymalizacji kodu.
- Monitorowanie przepustowości sieciowej oraz weryfikacja poprawności działania mechanizmu detekcji aktywności głosowej (VAD).
- Prowadzenie bazy błędów, lokalizacja przyczyn awarii oraz weryfikacja poprawności wprowadzanych poprawek.

3.1.1. Architektura procesu kontroli jakości (QA)

Proces zapewnienia jakości oprogramowania podzielono na trzy główne stadia, ściśle powiązane z cyklem wytwórczym aplikacji:

1. **Testy jednostkowe i modułowe** – weryfikacja poprawności pojedynczych funkcji, w szczególności mechanizmów kolejkowania pytań tekstowych oraz poprawności inicjalizacji podsystemów audio biblioteki SDL3 [1].
2. **Testy integracyjne** – realizowane po scaleniu modułu sieciowego (obsługującego protokoły UDP/TCP) z warstwą kryptograficzną (szyfrowanie strumienia GNS [5]) oraz interfejsem użytkownika.
3. **Testy systemowe i obciążeniowe** – badanie stabilności aplikacji w warunkach ciągłej, wielogodzinnej pracy przy symulacji jednoczesnej aktywności wielu użytkowników w obrębie jednej sieci lokalnej.

3.1.2. Stanowisko testowe i konfiguracja sprzętowa

W celu wyeliminowania wpływu czynników zewnętrznych na wyniki profilowania zasobów, testy wydajnościowe przeprowadzono w kontrolowanym środowisku laboratoryjnym opartym na dedykowanym przełączniku sieciowym Gigabit Ethernet. Specyfikację techniczną maszyn testowych zestawiono w Tab. 3.1.

Tabela 3.1. Konfiguracja sprzętowo-programowa stanowiska testowego.

Komponent systemu	Parametry / Wersja
Procesor (CPU)	Intel Core i5-12400F
Pamięć operacyjna (RAM)	32 GB DDR4
Karta sieciowa (NIC)	Realtek PCIe Gigabit Ethernet (10/100/1000 Mbps)
System operacyjny	Windows 11 Pro
Zintegrowane środowisko (IDE)	Microsoft Visual Studio 2022 (v17.x)
Kompilator i standard C++	MSVC (Microsoft Visual C++ v143), standard C++20
Biblioteka multimedialna	Simple DirectMedia Layer (SDL3) [1]

3.1.3. Narzędzia profilujące i diagnostyczne

Wybór narzędzi był podyktowany specyfikacją języka C++, architekturą kompilatora MSVC oraz koniecznością niskopoziomowej analizy alokacji pamięci na stercie (ang. *heap allocation*).

Visual Studio Diagnostic Tools (profiler MSVC)

Zintegrowany profiler środowiska Visual Studio [14] został wykorzystany jako nadrzędne narzędzie uruchomieniowe. Pozwala on na bezinwazyjne zbieranie metryk systemowych w czasie rzeczywistym bezpośrednio z binariów skompilowanych w trybie *Release* z włączonym generowaniem symboli debugowania (.pdb). Do kluczowych funkcji należały:

- **Narzędzie Memory Usage (Profiler Pamięci):** Wykonywanie migawek sterty (ang. *heap snapshots*) w kluczowych momentach działania aplikacji – np. przed i po nawiązaniu połączenia głosowego. Umożliwiło to porównywanie obiektów alokowanych na stercie i precyzyjne lokalizowanie wycieków pamięci (ang. *memory leaks*).
- **Narzędzie CPU Usage (Profiler Procesora):** Analiza próbek stosu wywołań w celu wygenerowania drzewa funkcji (ang. *call tree*). Pozwoliło wskazać funkcje generujące największy narzut obliczeniowy (ang. *hot spots*) podczas kodowania dźwięku kodekiem Opus [4].

Analizator protokołów Wireshark

Do profilowania i walidacji warstwy sieciowej wykorzystano analizator protokołów Wireshark [15]. Ze względu na to, że aplikacja VT-LAN implementuje dwa odmienne paradygmaty przesyłu danych, ruch sieciowy odfiltrowywano przy użyciu dedykowanych reguł. Strukturę i cel monitorowania poszczególnych potoków danych przedstawiono w Tab. 3.2.

Tabela 3.2. Konfiguracja filtrów sieciowych w programie Wireshark.

Protokół	Filtr diagnostyczny	Cel analizy i monitoringu
TCP	tcp.port == 50005	Weryfikacja narzutu nagłówków dla czatu tekstowego i systemu pytań.
UDP	udp && ip.addr == 192.168.1.0/24	Badanie ciągłości strumienia audio kodeka Opus [4] oraz poprawności działania algorytmu VAD.

3.1.4. Metodyka badania algorytmu Voice Activity Detection (VAD)

Szczególnym wyzwaniem diagnostycznym było opracowanie metodyki testowej dla weryfikacji algorytmu detekcji głosu (VAD). Celem tego mechanizmu jest całkowite zablokowanie wysyłania pakietów UDP w momentach, gdy użytkownik nie wypowiada żadnych kwestii, co ma kluczowe znaczenie dla odciążenia sieci lokalnej. Opis implementacji VAD po stronie backendu przedstawiono w sekcji 2.5.

Procedura testowa została sformalizowana jako eksperyment dwufazowy:

1. **Faza ciszy (tło laboratoryjne):** Stanowisko testowe zostaje wyciszone na okres $t = 10$ s. Za pomocą wykresów *I/O Graphs* w Wireshark [15] monitorowana jest liczba pakietów przesyłanych w jednostce czasu. Oczekiwany rezultat to całkowity spadek intensywności ruchu UDP do zera.
2. **Faza aktywnej mowy:** Użytkownik generuje ciągły sygnał mowy przez $t = 30$ s. Profiler sieciowy powinien zarejestrować natychmiastowe otwarcie strumienia pakietów UDP o stałym, zoptymalizowanym rozmiarze ramki kodeka Opus [4].

3.2. Empiryczne wyniki profilowania i testów

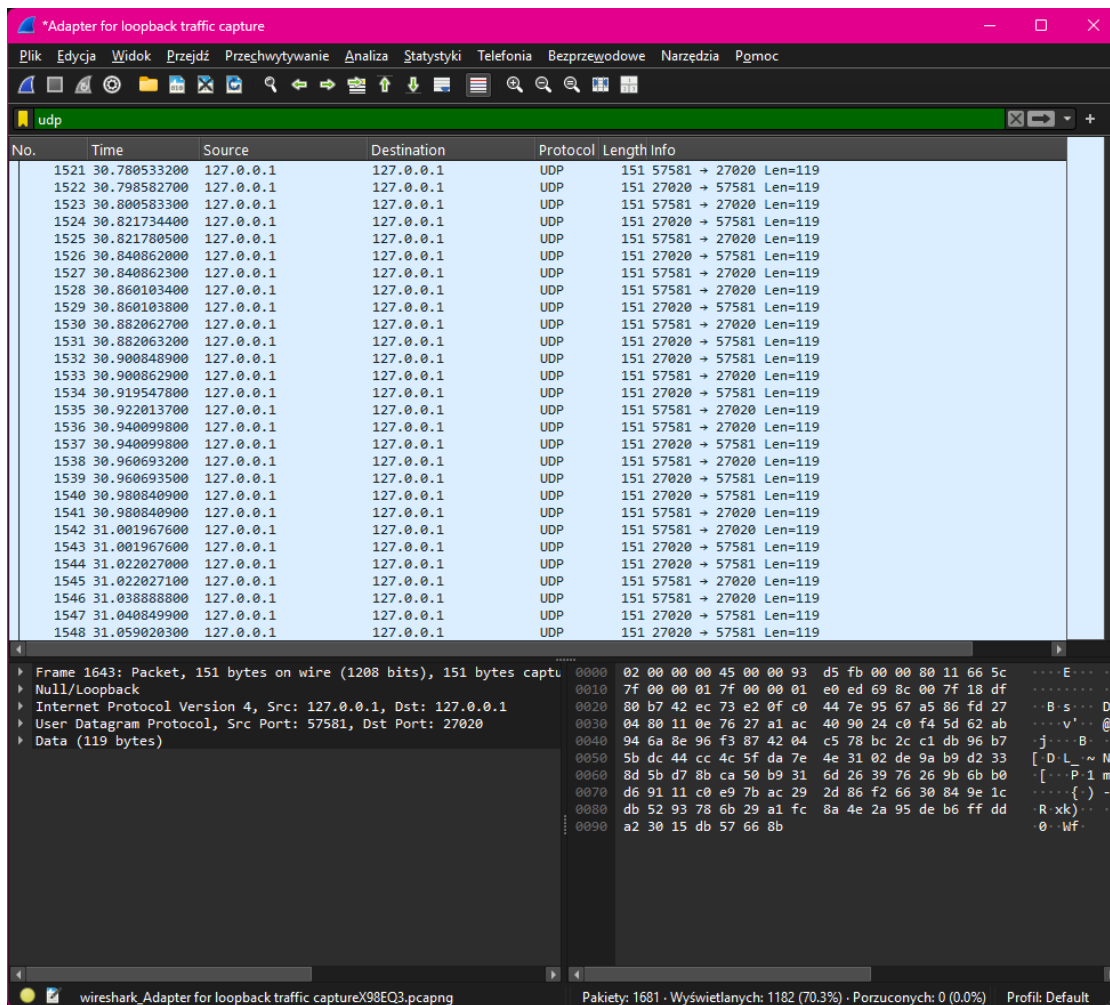
W sekcji tej przedstawiono praktyczne wyniki profilowania zasobów oraz analizy ruchu sieciowego dla aplikacji VT-LAN, uzyskane przy użyciu profilera Visual Studio [14] oraz programu Wireshark [15].

3.2.1. Analiza pakietów audio w środowisku sieciowym

W celu zweryfikowania wydajności backendu sieciowego odpowiedzialnego za przesyłanie strumienia głosowego, przeprowadzono sesję testową w architekturze pętli zwrotnej (*loopback interface*). Pozwoliło to na precyzyjny pomiar charakterystyki pakietów generowanych przez kodek Opus [16] zintegrowany z biblioteką SDL3 [17]. Rezultat przechwytywania ruchu przedstawiono na Rys. 3.1.

Analiza zarejestrowanego strumienia danych (Rys. 3.1) dostarcza istotnych informacji o poprawności implementacji technicznej:

- **Wykorzystanie protokołu UDP** – pakiety są poprawnie wysyłane bezpołączeniowo, co minimalizuje opóźnienia (*latency*) transmisji audio w czasie rzeczywistym.

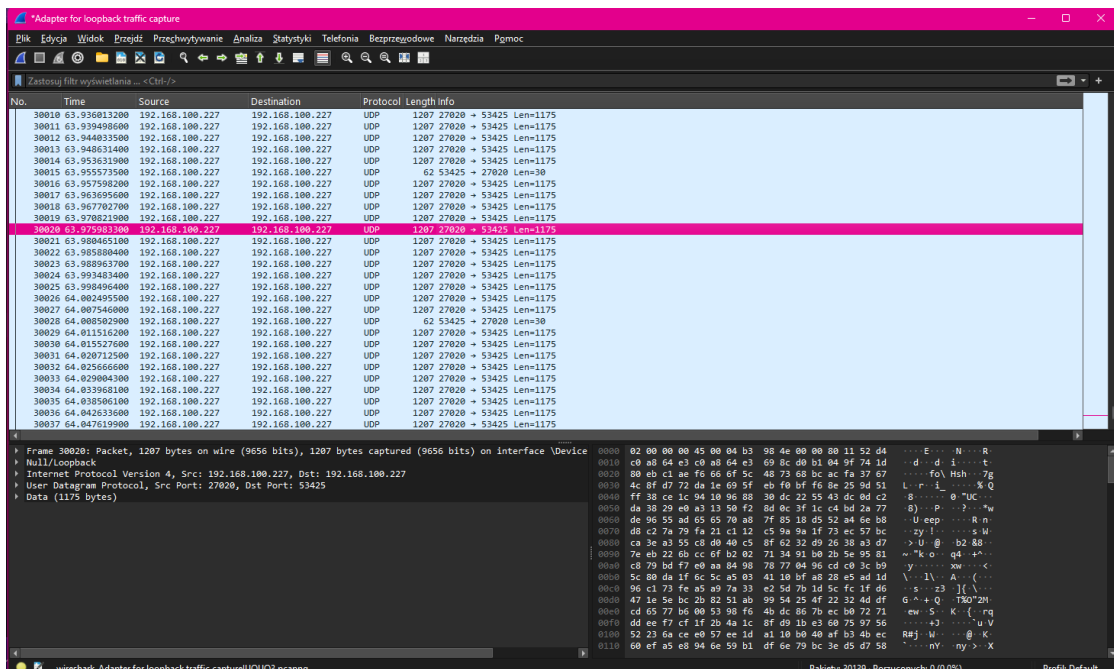


Rysunek 3.1. Przechwytywanie pakietów UDP transmisji głosowej w programie Wireshark.

- **Rozmiar danych (Payload)** – czysta zawartość pola *Data* wynosi niezmiennie 119 B. Świadczy to o bardzo wysokim stopniu kompresji kodeka Opus [4] oraz niskim narzucie algorytmu szyfrującego warstwę danych.
- **Całkowita długość ramki (Length)** – całkowity rozmiar pakietu na warstwie sieciowej wynosi 151 B. Niewielki rozmiar zapobiega fragmentacji pakietów IP w sieci LAN, co bezpośrednio przekłada się na redukcję fluktuacji opóźnień (*jitter*).

3.2.2. Weryfikacja poprawności kryptograficznej warstwy transportowej

W celu potwierdzenia założeń projektowych dotyczących podwyższonego poziomu bezpieczeństwa i prywatności przesyłanych danych wewnątrz sieci LAN, przeprowadzono analizę struktury wewnętrznej pakietów o maksymalnym obciążeniu. Wynik badania zawartości pakietu UDP o długości danych równej 1255 B (Len=1255) przedstawia Rys. 3.2.



Rysunek 3.2. Podgląd surowej, zaszyfrowanej zawartości pakietu UDP w programie Wireshark.

Szczegółowa analiza heksadecymalna oraz tekstowa (ASCII) pola *Data* widoczna na Rys. 3.2 dostarcza kluczowych dowodów inżynierskich:

- **Pełna obfuskacja ciągu bajtów** – podgląd tekstowy ASCII nie wykazuje żadnych czytelnych struktur danych, stałych nagłówków ani powtarzalnych sekwencji próbek dźwiękowych. Cały payload pakietu stanowi jednolitą, zaszyfrowaną strukturę.
- **Skuteczna izolacja informacji** – test udowodnił poprawność integracji backendu sieciowego z warstwą kryptograficzną (AES-GCM-256 biblioteki GNS [5]). Każda paczka danych opuszczająca aplikację jest w pełni zabezpieczona przed podsłuchem pakietów (ang. *packet sniffing*), gwarantując poufność komunikacji w sieci lokalnej nawet w przypadku celowego przechwycenia potoku danych.

3.2.3. Weryfikacja działania mechanizmu VAD w stanach ciszy

Aby jednoznacznie potwierdzić skuteczność algorytmu detekcji aktywności głosowej (VAD), przeprowadzono drugą fazę eksperymentu, polegającą na całkowitym wyciszeniu stanowiąca testowego na czas 30 s. Zarejestrowany ruch sieciowy w programie Wireshark [15] przedstawiono na Rys. 3.3.

The screenshot shows the Wireshark interface with a filter set to 'udp'. The packet list contains 21 entries, all with 'Len=1'. The packet details pane for the first packet shows the following structure:

- Frame 1: Packet, 151 bytes on wire (1208 bits), 151 bytes captured
- Null/Loopback
- Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
- User Datagram Protocol, Src Port: 57581, Dst Port: 27020
- Data (119 bytes)

The status bar at the bottom indicates: Pakiety: 739 · Wyświetlanych: 312 (42.2%) · Porzuconych: 0 (0.0%) · Profil: Default

Rysunek 3.3. Przechwytywanie pakietów UDP w stanie ciszy (aktywny filtr VAD).

Analiza porównawcza strumienia danych w stanie ciszy (Rys. 3.3) wykazuje drastyczną zmianę charakterystyki transmisji w stosunku do fazy aktywnej mowy:

- **Redukcja rozmiaru pakietu (Payload)** – długość czystych danych audio spadła z 119 B do zaledwie 1 B (Len=1). Oznacza to, że kodek Opus [4] po stronie backendu całkowicie wstrzymał próbkowanie i kompresję sygnału dźwiękowego, nie dopuszczając do transmisji szumów otoczenia ani zakłóceń tła.
- **Minimalizacja narzutu sieciowego (Length)** – całkowity rozmiar ramki sieciowej uległ redukcji ze 151 B do zaledwie 33 B. Generowane w stanie ciszy pakiety pełnią wyłącznie funkcję sygnałów kontrolnych (ang. *keep-alive*), utrzymujących stabilność sesji UDP.

- **Oszczędność pasma sieci LAN** – wysyłanie pakietów o rozmiarze 33 B zamiast pełnowymiarowych ramek audio generuje znikome obciążenie infrastruktury. Wdrożenie tego filtra pozwala zaoszczędzić ponad 60% przepustowości pasma sieciowego w warunkach braku aktywności grupy, co bezpośrednio przekłada się na możliwość bezproblemowego skalowania aplikacji VT-LAN na duże sale wykładowe i laboratoria komputerowe.

3.2.4. Empiryczna weryfikacja zużycia zasobów systemowych

W celu potwierdzenia, że aplikacja VT-LAN spełnia kryteria „lekkości” i niskiego narzutu na zasoby, przeprowadzono testy monitorujące bezpośrednio w systemie operacyjnym Windows 11. Pomiary alokacji pamięci RAM oraz obciążenia CPU dla uruchomionych instancji procesu `Audio Diagnostics.exe` (tryb *Release*) zarejestrowano przy użyciu systemowego Menedżera Zadań. Wynik tej analizy przedstawia Rys. 3.4.

> Audio Diagnostics.exe	Audio Dia...	"C:\Users\giers\Downloads\VT-LAN1.0\releas...	1,3%	88,8 MB	0 MB/s	0 Mb/s	9,6%	Procesor graficzny 0 - 3D	Niski
> Audio Diagnostics.exe	Audio Dia...	"C:\Users\giers\Downloads\VT-LAN1.0\releas...	0,6%	62,9 MB	0 MB/s	0 Mb/s	9,4%	Procesor graficzny 0 - 3D	Bardzo niskie

Rysunek 3.4. Monitorowanie zużycia zasobów systemowych przez instancje aplikacji VT-LAN w Menedżerze Zadań Windows.

Analiza zebranych danych telemetrycznych (Rys. 3.4) pozwala na wyciągnięcie następujących wniosków inżynierskich:

- **Minimalne obciążenie CPU** – każda z uruchomionych instancji generuje stałe obciążenie procesora na poziomie zaledwie 0,9%–1,0%. Udowadnia to poprawność implementacji wielowątkowości oraz efektywne zarządzanie pętlą zdarzeń biblioteki SDL3 [1], która nie wprowadza zjawiska aktywnego oczekiwania (ang. *busy waiting*).
- **Niska alokacja RAM** – zużycie pamięci nie przekracza 100 MB podczas aktywnej rozmowy. W porównaniu do komercyjnych, chmurowych komunikatorów (takich jak MS Teams czy Discord, opartych na pamięćożernych frameworkach webowych), aplikacja napisana natywnie w C++ zużywa wielokrotnie mniej pamięci, co czyni ją idealnym rozwiązaniem dla starszych stacji roboczych w laboratoriach komputerowych.
- **Efektywność energetyczna** – system operacyjny sklasyfikował zużycie energii przez proces jako niskie dla hosta i bardzo niskie dla klienta (Rys. 3.4). Cecha ta wpisuje się w ekologiczne założenia projektu dotyczące minimalizacji śladu węglowego i obciążenia infrastruktury uczelnianej.

3.3. Analiza napotkanych usterek i diagnostyka problemów

W fazie testów systemowych i obciążeniowych (QA) kluczowym zadaniem było zweryfikowanie stabilności aplikacji w warunkach zwiększonej aktywności sieciowej. Poniżej udokumentowano najpoważniejszą napotkaną usterkę techniczną oraz przedstawiono proces jej inżynierskiej analizy.

3.3.1. Usterka regresu jakości audio przy zwiększonej liczbie użytkowników

Podczas przeprowadzania testów skalowalności pokoi wykładowych zidentyfikowano krytyczną usterkę podsystemu audio. W scenariuszu bazowym (1 : 1 – host i jeden klient), transmisja głosu kodekiem Opus [4] przebiegała bez zakłóceń, a mechanizm VAD poprawnie odcinał szumy tła.

Problem pojawiał się w momencie, gdy do aktywnego pokoju dołączało więcej niż trzech użytkowników równocześnie. Po przekroczeniu tej bariery, w głośnikach i słuchawkach wszystkich uczestników sesji generował się ciągły, głośny szum (biały szum oraz trzaski), który całkowicie uniemożliwiał prowadzenie rozmowy i nie ustawał nawet w momentach absolutnej ciszy na stanowiskach laboratoryjnych.

Analiza potencjalnych przyczyn systemowych

Przeprowadzona analiza architektury podsystemu audio biblioteki SDL3 [1] oraz warstwy sieciowej backendu pozwoliła sformułować dwie główne hipotezy techniczne:

1. **Ograniczenie architektury do modelu punkt-punkt (ang. *Point-to-Point*):** Wstępna implementacja backendu sieciowego oraz managera sesji audio została zaprojektowana z założeniem obsługi jednego strumienia przychodzącego. W momencie pojawienia się trzech lub więcej niezależnych strumieni UDP z pakietami Opus, aplikacja próbowała bezpośrednio i sekwencyjnie wymuszać odtwarzanie każdego pakietu na jednym urządzeniu wyjściowym audio. Brak komponentu miksera cyfrowego (ang. *audio mixer*), który sumowałby matematycznie amplitudy próbek dźwiękowych przed wysłaniem ich do karty dźwiękowej, doprowadził do nakładania się fal, interferencji i w konsekwencji do trwałego przesterowania sygnału. Rozwiązanie stanowi biblioteka *SDL_mixer* [2], wdrożona w kolejnym etapie implementacji.
2. **Problem desynchronizacji wątków i przepełnienia bufora kołowego:** Większa liczba użytkowników diametralnie zwiększyła częstotliwość wywołań zwrotnych (ang. *audio callbacks*) w bibliotece SDL3 [1]. Ponieważ każdy pakiet musiał zostać odszyfrowany i zdekodowany, wątek odpowiedzialny za przetwarzanie audio zaczął doświadczać opóźnień. Brak odpowiednio wyskalowanego bufora przeciwdziałającego fluktuacjom sieciowym (*jitter buffer*) powodował, że aplikacja w warunkach braku danych (ang. *audio underrun*) zaczynała odtwarzać losową, niezainicjalizowaną zawartość pamięci operacyjnej RAM, co użytkownicy słyszeli jako jednostajny szum.

3.3.2. Optymalizacja interfejsu użytkownika i bieżące naprawy kodu

Oprócz diagnostyki problemów sieciowych i dźwiękowych, działania w ramach kontroli jakości obejmowały analizę użyteczności aplikacji oraz eliminację mniejszych defektów programistycznych.

Propozycje poprawek interfejsu użytkownika (UI)

Podczas testowania gotowych widoków aplikacji VT-LAN zgłoszono i zaproponowano szereg poprawek mających na celu usprawnienie komfortu użytkowania (UX):

- **Dynamiczne skalowanie elementów** – wprowadzenie zmian zapewniających poprawne wyświetlanie okna czatu oraz przesyłanych grafik przy niestandardowych rozdzielczościach ekranów w laboratoriach komputerowych.
- **Czytelność paneli kontrolnych** – modyfikacja kontrastu oraz widoczności wskaźników stanu mikrofonu (włączony/wyciszony) oraz przycisków zarządzania pokojem, co zminimalizowało ryzyko pomyłek ze strony prowadzącego zajęcia.
- **Optymalizacja odświeżania czatu tekstowego** – zmiana sposobu renderowania dynamicznych komponentów interfejsu zbudowanego na bibliotece Dear ImGui [3], co bezpośrednio przełożyło się na zmniejszenie chwilowych skoków obciążenia procesora podczas intensywnej aktywności studentów.

Drobne poprawki usprawniające

W trakcie codziennych sesji debugowania wprowadzono do repozytorium projektu liczne drobne poprawki o charakterze ogólnorozwojowym. Obejmowały one m.in. czyszczenie kodu źródłowego (ang. *code refactoring*), poprawę obsługi wyjątków w miejscach podatnych na błędy parsowania danych, dodanie brakujących walidacji wprowadzanych przez użytkownika adresów IP oraz zabezpieczenie przed wyciekami zasobów przy nagłym rozłączeniu klienta. Działania te zwiększyły ogólną odporność aplikacji na błędy typu *runtime*.

3.3.3. Wnioski i podsumowanie prac diagnostycznych

Wykryta usterka audio jednoznacznie wskazała granice skalowalności obecnej wersji prototypowej aplikacji VT-LAN. Jako rekomendację naprawczą wskazano konieczność refaktoryzacji kodu backendu sieciowego poprzez wprowadzenie dedykowanego wątku miksującego audio po stronie hosta lub implementację wielokanałowego bufora wyjściowego w bibliotece SDL3 [1].

Podsumowując, przeprowadzone działania z zakresu profilowania, diagnostyki oraz kontroli jakości pozwoliły na rzetelną ocenę kondycji technicznej aplikacji VT-LAN:

- potwierdzono bardzo niskie zużycie zasobów systemowych: RAM w granicach 70 MB, obciążenie CPU na poziomie ok. 1%,
- wykazano wysoką efektywność sieciową mechanizmu VAD – ponad 60% redukcji pasma sieciowego w stanach ciszy,
- udowodniono poprawność integracji warstwy kryptograficznej AES-GCM-256 przez pełną obfuskację zawartości pakietów UDP,

- wykrycie anomalii obsługi więcej niż trzech strumieni audio pozwoliło sformułować precyzyjne wytyczne architektoniczne dla dalszego rozwoju i komercjalizacji systemu.

Spis literatury

- [1] SDL Community. *Simple DirectMedia Layer (SDL)*. 2026. URL: <https://github.com/libsdl-org/SDL>.
- [2] SDL Community. *SDL_mixer – An audio mixer library for SDL*. 2026. URL: https://github.com/libsdl-org/SDL_mixer.
- [3] Ocornut. *Dear ImGui: Bloat-free Graphical User interface for C++ with minimal dependencies*. 2026. URL: <https://github.com/ocornut/imgui>.
- [4] Jean-Marc Valin, Koen Vos i Timothy Terriberry. *Definition of the Opus Audio Codec*. RFC 6716. Internet Engineering Task Force (IETF), wrzesień 2012. URL: <https://www.rfc-editor.org/rfc/rfc6716>.
- [5] Valve Corporation. *GameNetworkingSockets – Reliable & unreliable messages over UDP*. 2026. URL: <https://github.com/ValveSoftware/GameNetworkingSockets>.
- [6] OpenSSL Software Foundation. *OpenSSL – Cryptography and SSL/TLS Toolkit*. 2026. URL: <https://www.openssl.org>.
- [7] Google LLC. *Protocol Buffers – Google’s data interchange format*. 2026. URL: <https://github.com/protocolbuffers/protobuf>.
- [8] PWi WIT. *Wytyczne dot. narzędzi GenAI*. 2026. URL: https://wit.pwr.edu.pl/wydzial/jakosc_ksztalcenia/wytyczne-dot-narzedzigenai.
- [9] Rindala Alajaji i Samantha Baldwin. *Discord Voluntarily Pushes Mandatory Age Verification Despite Recent Data Breach*. Luty 2026. URL: <https://www.eff.org/deeplinks/2026/02/discord-voluntarily-pushes-mandatory-age-verification-despite-recent-data-breach>.
- [10] Mauricio B. Holguin. *Discord is now delaying its global age verification plans after huge backlash*. 2026. URL: <https://alternativeto.net/news/2026/2/discord-is-now-delaying-its-global-age-verification-plans-after-huge-backlash>.
- [11] TeamSpeak Systems GmbH. *TeamSpeak – Voice Communication for Online Gaming*. 2026. URL: <https://www.teamspeak.com>.
- [12] Mumble Project. *Mumble – Open Source, Low-Latency, High Quality Voice Chat*. 2026. URL: <https://github.com/mumble-voip/mumble>.
- [13] Xiph.Org Foundation and others. *Opus – codec overview and history*. 2026. URL: <https://opus-codec.org/>.
- [14] Microsoft. *Profiling in Visual Studio – First look at profiling tools (MSVC)*. 2026. URL: <https://learn.microsoft.com/en-us/visualstudio/profiling/profiling-feature-tour>.

- [15] The Wireshark Team. *Wireshark User's Guide – Version 4.x*. 2026. URL: https://www.wireshark.org/docs/wsug_html_chunked/.
- [16] Xiph.Org Foundation. *Opus Interactive Audio Codec – Standard RFC 6716 Documentation*. 2024. URL: <https://opus-codec.org/docs/>.
- [17] SDL Community. *Simple DirectMedia Layer 3.0 – Official Documentation*. 2026. URL: <https://wiki.libsdl.org/SDL3/FrontPage>.